

Ejercicios sobre complejidad

A partir de los siguientes algoritmos.

- Determina cuántas operaciones se realizarán en cada caso.
- Determina cuántas operaciones se realizarán por iteración.
- Determina la complejidad del algoritmo.

Algoritmo 1

```
int funcion(int n) {  
    int x = 5;           // Línea 1  
    int y = x + 3;       // Línea 2  
    int z = x * y;       // Línea 3  
    int resultado = z - 10; // Línea 4  
    return resultado;    // Línea 5  
}
```

Algoritmo 2

```
for (int i = 0; i < n; i += 2) {  
    cout << i << endl;  
}
```

Algoritmo 3

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < 5; j++) {  
        cout << i << " " << j << endl;  
    }  
}
```

Algoritmo 4

```
// Bucle 1  
for (int i = 0; i < n; i++) {  
    cout << i << endl;  
}  
  
// Bucle 2  
for (int j = 0; j < n; j++) {  
    cout << j << endl;  
}  
  
// Bucle 3  
for (int k = 0; k < n; k++) {  
    cout << k << endl;  
}
```

Algoritmo 5

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
            cout << i << " " << j << " " << k << endl;
        }
    }
}
```

Algoritmo 6

```
int** matriz = new int*[n];
for (int i = 0; i < n; i++) {
    matriz[i] = new int[n];
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        matriz[i][j] = i + j;
    }
}
```

Algoritmo 7

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        cout << i << " " << j << endl;
    }
}
```

Algoritmo 8

```
int arr[n];
int target;

for (int i = 0; i < n; i++) {
    //
    for (int j = 0; j < n; j++) {
        if (arr[j] == target) {
            break;
        }
    }
}
```

Algoritmo 9

```
int i = n;
while (i > 1) {
    cout << i << endl;
    i = i / 2;
}
```

Algoritmo 10

```
for (int i = 0; i < n; i++) {
    int j = 1;
    while (j < n) {
        cout << i << " " << j << endl;
        j = j * 2;
    }
}
```

Algoritmo 11

```
int search(int arr[], int n, int target) {
    /*
    Búsqueda
    */
    int salto = (int)sqrt(n);

    //
    int prev = 0;
    while (arr[min(salto, n) - 1] < target) {
        prev = salto;
        salto += (int)sqrt(n);

        // Si llegamos al final sin encontrar
        if (prev >= n) {
            return -1;
        }
    }

    // Búsqueda en el bloque
    while (arr[prev] < target) {
        prev++;

        // Si llegamos al siguiente bloque o al final
        if (prev == min(salto, n)) {
            return -1;
        }
    }

    // Verificar si encontramos el elemento
    if (arr[prev] == target) {
        return prev;
    }
}
```

```

    return -1;
}

```

Algoritmo 12

```

#include <stdio.h>
#include <stdlib.h>

/* ----- */
/* merge(): fusiona dos subarreglos adyacentes ya ordenados */
/*   A[izq .. med]      (izquierdo) */
/*   A[med+1 .. der]    (derecho) */
/* usando un arreglo auxiliar 'tmp'. */
/* ----- */
void merge(int A[], int tmp[], int izq, int med, int der) {
    int i = izq;      /* recorre el subarreglo izquierdo */
    int j = med + 1;  /* recorre el subarreglo derecho */
    int k = izq;      /* recorre el arreglo de salida */

    /* (1) mezcla comparando cabeza contra cabeza */
    while (i <= med && j <= der) { /* <-- ciclo principal */
        if (A[i] <= A[j])
            tmp[k++] = A[i++];
        else
            tmp[k++] = A[j++];
    }

    /* (2) copia el remanente del lado izquierdo (si quedó) */
    while (i <= med)
        tmp[k++] = A[i++];

    /* (3) copia el remanente del lado derecho (si quedó) */
    while (j <= der)
        tmp[k++] = A[j++];

    /* (4) vuelca el segmento fusionado de tmp de regreso a A */
    for (k = izq; k <= der; k++)
        A[k] = tmp[k];
}

/* ----- */
/* merge_sort_iterativo(): version NO recursiva (bottom-up). */
/* En lugar de dividir con recursion, va fusionando bloques de */
/* tamaño 'ancho' que se duplica en cada pasada: 1, 2, 4, 8, ... */
/* ----- */
void merge_sort_iterativo(int A[], int n) {
    int *tmp = (int *) malloc(n * sizeof(int));
    if (tmp == NULL) return;

    int ancho, izq;

    /* CICLO EXTERNO: ancho de los bloques a fusionar */
    for (ancho = 1; ancho < n; ancho = 2 * ancho) {

        /* CICLO INTERNO: recorre el arreglo por pares de bloques */

```

```

    for (izq = 0; izq < n; izq = izq + 2 * ancho) {
        int med = izq + ancho - 1;
        int der = izq + 2 * ancho - 1;

        if (med >= n - 1) continue;      /* no hay bloque derecho */
        if (der > n - 1) der = n - 1;    /* recorta el ultimo bloque */

        merge(A, tmp, izq, med, der);
    }
}

free(tmp);
}

int main(void) {
    int A[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(A) / sizeof(A[0]);

    merge_sort_iterativo(A, n);

    for (int i = 0; i < n; i++)
        printf("%d ", A[i]);
    printf("\n");
    return 0;
}

```

Análisis de funciones

Ordena la complejidad de las siguientes funciones de la **menor (más rápida)** a la **mayor (más lenta)**

- a) $5n + 100$
- b) $0.001n^2$
- c) n
- d) 1000
- e) $n + \log n$
- f) $100n$

Simplificación

Simplifica las siguientes expresiones a notación Big-O:

- a) $5n^2 + 3n + 100 = \underline{\hspace{2cm}}$
- b) $n^2/2 + n/2 = \underline{\hspace{2cm}}$
- c) $10n + 500 = \underline{\hspace{2cm}}$
- d) $0.01n^3 + 1000n^2 = \underline{\hspace{2cm}}$
- e) $\log n + n + n \log n = \underline{\hspace{2cm}}$

Términos Dominantes

Para cada expresión, identifica el **término dominante**:

- a) $T(n) = 3n^2 + 5n + 10$
Término dominante: _____
- b) $T(n) = 100n + 50 \log n + 200$
Término dominante: _____
- c) $T(n) = n^3 + n^2 \log n$
Término dominante: _____
- d) $T(n) = 2^n + n^{10}$
Término dominante: _____

Completa el siguiente cuadro, proporciona el tamaño de n para que el tiempo de ejecución corresponda al indicado y con la complejidad específica.

Cada
operación toma
1
microsegundo

Andrés García Floriano

1-1 Comparison of running times

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

log n=1sec
log n=1,000,000 micro secs
n=?

Andrés García Floriano

log n=60 sec
log n=60,000,000 micro secs
n=?

Andrés García Floriano

Chapter 1 The Role of Algorithms in Computing

	1 second	1 minute	1 hour	1 day	1 month	1 year	1 century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							